

AIQT CLI

AIQT MVP Specification v0.6

Lean Workflow Engine

Field	Value
Product	AIQT CLI
Specification	MVP Specification
Version	v0.6
Name	Lean Workflow Engine
Status	Implementation-aligned baseline
Primary User	Human project owner using AI coding agents
Primary Agent Target	Claude Code or equivalent coding agent
Primary Design Principle	State first, documents second

1. v0.6 Change Summary

- Clarifies that canonical JSON files use camelCase field names. Snake_case examples are removed from schema examples.
- Separates AIQT file schema version from npm package version. Canonical files use AIQT_SCHEMA_VERSION.
- Adds Assumption, QualityExpectations, and Integrations as explicit project model concepts.
- Defines runlog health behavior at product level: malformed runlog lines warn but do not corrupt operational state.
- Clarifies that Milestone 1 uses non-interactive init flags only; interactive project intake belongs to aiqt update.
- Preserves the v0.5 direction: no file-per-milestone model and no markdown-first workflow.

2. Product Objective

AIQT is a local CLI workflow engine that helps a human project owner plan, structure, hand off, review, and evolve software projects built with AI coding agents. The MVP converts a rough project idea into a controlled, resumable, agent-ready implementation workflow without creating excessive documentation overhead.

Idea -> Structured project state -> Work graph -> Agent handoff packet -> Checkpoint -> Next work unit

AIQT is not a full project management platform. It is a local workflow state engine for creating reliable AI-agent execution context.

3. Core Problem

- AI agents often receive too little context or too much unstructured context.
- Project information is frequently scattered across markdown files and chat history.
- Acceptance criteria are often vague or missing.
- There is no stable workflow state after partial implementation.
- Human-readable documentation and agent-executable instructions are often mixed together.

AIQT solves this by maintaining compact structured state and generating bounded handoff packets for one current work unit.

4. MVP Design Principles

4.1 State First, Documents Second

AIQT stores project knowledge as structured state. Markdown is an optional generated view, never the primary database.

4.2 Agent Handoff Over Human Readability

Agent-facing outputs must prioritize explicit objectives, scope, constraints, acceptance criteria, validation commands, and required output format. Human-readable documents are generated separately through export commands.

4.3 Dynamic Work Graph

AIQT models implementation as a graph. A project may have one milestone, three milestones, thirty milestones, nested work units, blocked work, and replanned work. Milestone count is never assumed.

4.4 Lean CLI Surface

Commands map to core workflow actions, not every possible granular planning action.

4.5 Controlled AI Execution

AIQT does not ask an agent to build the full app by default. Each handoff is bounded to one current work unit with explicit non-goals and validation expectations.

5. MVP Scope

Included	Excluded
Project initialization	Full SaaS backend
Project intake and refinement	Multi-user collaboration
Canonical project state storage	Real-time agent orchestration
Dynamic work graph generation	Cloud synchronization
Agent handoff packet generation	GUI
Checkpoint capture and workflow review	Full issue tracker replacement
Human-readable export on demand	Automatic code modification

6. Core Workflow

Capture -> Plan -> Handoff -> Execute -> Checkpoint -> Review -> Continue

1. The user starts or updates project context.
2. AIQT stores canonical structured state.
3. AIQT generates or refreshes the work graph.
4. The user requests the next implementation step.
5. AIQT creates a bounded agent handoff packet.
6. The coding agent performs the work outside AIQT.
7. The user checkpoints the result.
8. AIQT updates state and recommends the next action.

7. Canonical File Model

```
.aiqt/
  project.json
  state.json
  runlog.jsonl
  exports/
```

The source of truth is project.json plus state.json. The runlog is append-only execution history. Exports are generated views and are not required for normal workflow execution.

File	Responsibility
.aiqt/project.json	Durable project definition: objective, users, requirements, constraints, decisions, risks, assumptions, open questions, quality expectations, integrations.
.aiqt/state.json	Transient workflow state: current milestone, current work unit, work graph, statuses, dependencies, checkpoints, next command.
.aiqt/runlog.jsonl	Append-only event history for initialization, updates, planning, packet creation, checkpoints, reviews, exports, and status changes.
.aiqt/exports/	Optional generated markdown outputs only.

8. Canonical Naming and Versioning

- All canonical JSON files use camelCase field names.
- Any snake_case examples from older drafts are illustrative only and must not be used as file schema.
- AIQT_SCHEMA_VERSION identifies the canonical file schema version.
- package.json.version identifies the distributed npm package version and may change without changing file schema.

```
AIQT_SCHEMA_VERSION = "0.5.0"
```

```
project.json.version = AIQT_SCHEMA_VERSION
state.json.version = AIQT_SCHEMA_VERSION
```

9. Core Data Model

9.1 Project Model

```
{
  "version": "0.5.0",
  "project": {
    "id": "PROJECT-001",
    "name": "Project Name",
    "objective": "Primary project objective",
    "targetUsers": [],
    "preferredAgent": null,
    "existingRepositoryPath": null,
    "createdAt": "ISO_DATE",
    "updatedAt": "ISO_DATE"
  },
  "context": {
    "constraints": [],
    "nonGoals": [],
    "technologyPreferences": [],
    "businessRules": [],
    "architectureNotes": []
  },
  "requirements": [],
  "decisions": [],
  "risks": [],
  "assumptions": [],
  "openQuestions": [],
  "quality": {
    "acceptanceCriteriaRequired": true,
    "validationRequiredBeforeDone": true,
    "preferredValidationCommands": []
  },
  "integrations": {}
}
```

9.2 Assumption

```
{
  "id": "A001",
  "statement": "Assumption text",
  "reason": null,
  "source": "human | aiqt | agent | system",
  "status": "active | replaced | invalidated",
  "createdAt": "ISO_DATE",
  "updatedAt": "ISO_DATE"
}
```

9.3 Decision

```
{
  "id": "D001",
  "date": "YYYY-MM-DD",
  "decision": "Decision text",
  "reason": "Why this decision was made",
  "impact": "What this changes",
  "status": "proposed | decided | reversed | deferred"
}
```

9.4 Work Graph

```
{
  "milestones": [],
  "workUnits": [],
  "dependencies": []
}
```

9.5 Milestone

```
{
  "id": "M001",
  "title": "Milestone title",
  "objective": "What this milestone achieves",
  "status": "planned | ready | in_progress | blocked | done | cancelled",
  "workUnitIds": []
}
```

9.6 Work Unit

```
{
  "id": "WU001",
  "milestoneId": "M001",
  "title": "Work unit title",
  "objective": "Specific outcome",
  "scope": [],
  "outOfScope": [],
  "acceptanceCriteria": [],
  "dependencies": [],
  "status": "ready",
  "agentContextRefs": [],
  "suggestedFiles": [],
  "validationCommands": [],
  "createdAt": "ISO_DATE",
  "updatedAt": "ISO_DATE"
}
```

9.7 Checkpoint

```
{
  "id": "C001",
  "workUnitId": "WU001",
  "summary": "What changed",
  "completed": [],
  "notCompleted": [],
  "filesChanged": [],
  "issues": [],
  "validationResult": "passed | failed | partial | not_run",
  "acceptanceCriteriaResult": "passed | failed | partial | not_checked",
  "nextRecommendation": "Recommended next action"
}
```

10. Agent Handoff Packet

The agent handoff packet is the primary MVP output. It tells the coding agent what to do now, why it matters, what context is relevant, what not to touch, how success will be validated, and what output the agent should return.

```
# AGENT EXECUTION PACKET
## Role
You are implementing one bounded work unit for this project.
## Project Objective
<short project objective>
## Current Work Unit
ID: <workUnitId>
Title: <title>
Objective: <objective>
## Scope
- <in-scope item>
## Out of Scope
- <out-of-scope item>
## Relevant Context
- <only context needed for this work unit>
## Constraints
```

```

- <technical or product constraint>
## Acceptance Criteria
- <testable criterion>
## Suggested Files / Areas
- <file or area if known>
## Validation Commands
<command>
## Required Agent Output
Return: summary, files changed, validation performed, issues/risks, suggested next step.

```

11. CLI Command Surface

Command	Purpose
aiqt init	Create canonical .aiqt files.
aiqt update	Update durable project context, requirements, decisions, risks, assumptions, and open questions.
aiqt plan	Generate or refresh the dynamic work graph.
aiqt next	Generate the next agent handoff packet for one ready work unit.
aiqt checkpoint	Capture result of an agent execution cycle.
aiqt review	Run quality checks and suggest corrections.
aiqt export	Generate human-readable documentation from state.
aiqt status	Read current workflow state without mutation.

12. Status Model

Entity	Statuses
Project	draft, planned, in_progress, blocked, review, complete
Work Unit	planned, ready, in_progress, needs_review, done, blocked, replanned, cancelled
Milestone	planned, ready, in_progress, blocked, done, cancelled

13. Runlog Health Rules

- project.json and state.json are the operational source of truth.
- runlog.jsonl is append-only history.
- Missing or unreadable runlog.jsonl is invalid state.
- Malformed individual runlog lines should produce warnings and be exposed through runlogHealth, but should not prevent status/next from reading project.json and state.json.

14. Planning Rules

- Do not create one file per milestone.
- Do not assume a fixed number of milestones.
- Split work units by agent-executable outcomes.
- Keep work units small enough for one focused agent session.
- Every ready work unit must have acceptance criteria, out-of-scope boundaries, and validation instructions.

- Replanning is normal and must preserve history.
- Human documentation is generated only on demand.

15. MVP Acceptance Criteria

- Initialize a project with .aiqt/ state files.
- Store durable project context in project.json.
- Store workflow state and work graph in state.json.
- Append workflow events to runlog.jsonl.
- Generate a dynamic work graph with any number of milestones.
- Represent milestones and work units as structured records, not files.
- Generate bounded agent handoff packets.
- Capture checkpoint results and update work-unit status.
- Review missing criteria, blockers, circular dependencies, and oversized work units.
- Export human-readable markdown only on request.
- Resume workflow from saved state without losing context.

16. Implementation Priority

Priority	Build
1	Canonical State Engine: project.json, state.json, runlog.jsonl, state loading/saving, event append logic.
2	Project Update Engine: context, requirements, decisions, assumptions, risks, open questions.
3	Planning Engine: milestones, work units, dependencies, readiness validation.
4	Agent Packet Generator: bounded packet with acceptance criteria and validation commands.
5	Checkpoint Engine: checkpoint records and status transitions.
6	Review and Export Engines: quality checks and generated views.

17. Key Design Decision

AIQT behaves like a workflow state engine, not a markdown document generator.

Prioritize:

state -> graph -> packet -> checkpoint

Do not prioritize:

spec -> milestone file -> task file -> review file -> another markdown file